

5.4 *Potprogrami (funkcije) u C++*

- 5.4.1 Koncept globalnih i lokalnih podataka
- 5.4.2 Statički članovi
- 5.4.3 Pokazivači i reference kao argumenti
- 5.4.4 Preopterećenje funkcija
- 5.4.5 Rekurzivna funkcija
- 5.4.6 Pretprocesorska naredba `#include`
- 5.4.7 Standardne biblioteke funkcija

5.4 Potprogrami (funkcije) u C++

Potprogrami su izdvojene logičke cjeline unutar programa. Za razliku od Pascala, koji razlikuje procedure i funkcije, C/C++ poznaje samo funkcije. Funkcija je zamišljena tako da vrati vrijednost koja odgovara njenom tipu (pomoću naredbe *return*). Ako funkcija umjesto tipa ima oznaku *void*, tada ne vraća vrijednost, odnosno, predstavlja pandan proceduri.

Opći oblik funkcije :

```
<povratni tip ili void> ime_funkcije (<tip> par1, <tip> par2, ... ,<tip> par N)
{
    .
    .
    .
    <blok instrukcija>
    .
    .
    .
    return <povratna vrijednost>; // izostavlja se ako je funkcija tipa void
}
```

Razlikujemo formalne parametre (navode se u zaglavlju funkcije) i stvarne parametre (navode se kod poziva funkcije). Prilikom poziva funkcije formalni parametri poprimaju vrijednosti odgovarajućih stvarnih parametara.

Primjer :

```
#include <iostream.h>
float kvadrat (float x){
    return x * x;
}
void main () { // main je rezervirano ime
    float broj;
    float kvad = kvadrat (broj);
    cout << broj << " na kvadrat iznosi " << kvad << endl;
}
```

5.4.1 Koncept globalnih i lokalnih podataka

Globalni podaci dostupni su u svim dijelovima programa (osim u funkcijama gdje su definirani istoimeni lokalni podaci), dok su lokalni dostupni isključivo u okviru funkcije gdje su definirani.

Primjer :

```
#include <iostream.h>
int a,b;
float c,d; // a,b,c,d su globalne varijable
void main () {
    int e,f; //varijable e,f lokalne su za funkciju main
    char a,c // varijable a,c lokalne su za funkciju main i prekrivaju
              // istoimene globalne varijable

    a = 'A';
    c = 'B';
}
```

```

        b = 5;           // globalne varijable b,d dostupne su unutar funkcije main
        d = 3.14;
    }

```

5.4.2 Statički članovi

Statički članovi označavaju se pomoću ključne riječi *static*.

Statička lokalna varijabla ima doseg lokalne, a trajanje globalne varijable :

Primjer :

```

#include <iostream.h>
void funkcija_stat () {
    int a;
    static int b;
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    a = 5;
    b = 10;
}
void main () {
    cout << "1. poziv funkcije :" << endl;
    funkcija_stat();
    cout << "2. poziv funkcije :" << endl;
    funkcija_stat();
}

```

Daje slijedeći ispis kao rezultat :

```

1. poziv funkcije :
a = 6618616
b = 0
2. poziv funkcije :
a = 6618616
b = 10

```

Vrijednost statičke lokalne varijable *b* ostala je sačuvana do slijedećeg poziva funkcije.

5.4.3 Pokazivači i reference kao argumenti

Osim prijenosa argumenata po vrijednosti, moguće je koristiti pokazivače i reference kao argumente. U tom slučaju funkcija ne vrši obradu nad kopijama podataka nego nad originalnim podacima.

Primjer (pokazivači kao argumenti) :

```

#include <iostream h>
void zamjena (int *prvi, int *drugi){
    int pom = *prvi;
    *prvi = *drugi;
    *drugi = pom;
}
void main (){

```

```

    int a=5;
    int b=10;
    zamjena (&a,&b);
    cout << "a = " << a << endl; // 10
    cout << "b = " << b << endl; // 5
}

```

Primjer (reference kao argumenti) :

```

#include <iostream.h>
void zamjena (int &prvi, int &drugi) {
    int pom = prvi;
    prvi = drugi;
    drugi = pom;
}
void main () {
    int a=5;
    int b=10;
    zamjena (a,b);
    cout << "a = " << a << endl; // 10
    cout << "b = " << b << endl; // 5
}

```

5.4.4 Preopterećenje funkcija

Preopterećenje omogućuje korištenje istog imena za različite funkcije, pri čemu se te funkcije međusobno razlikuju po tipu argumenata u svom zaglavlju.

Primjer:

```

#include <iostream.h>
void funkcija (int n){
    cout << "n (int) = " << n << endl;
}
void funkcija (float n){
    cout << "n (float) = " << n << endl;
}
void main () {
    int a=5;
    float b=3.125;
    funkcija (a); //cjelobrojni parametar
    funkcija (b); //realni parametar
}

```

5.4.5 Rekurzivna funkcija

Rekurzija je svojstvo potprograma (funkcije) da može pozivati sam sebe.

Primjer (računanje n!) :

```

#include <iostream.h>
int fakto (int n){
    if (n ==1)        //sidreni uvjet

```

```

        return 1;
    else
        return n*fakto (n - 1);    //pojednostavljenje
}
void main () {
    cout << fakto (5) << endl;
}

```

Za realizaciju rekurzivne funkcije potrebno je osigurati slijedeće :

Sidreni izraz : $1! = 1$

Rekurzivnu formulu ili postupak : $n! = n * (n - 1)!$

Sidreni izraz omogućuje izlazak iz rekurzivne funkcije (tj. spriječava beskonačno pozivanje funkcije), a rekurzivna formula ili postupak trebaju pojednostaviti zadatak, odnosno, približiti ga rješenju zadanom kao sidreni izraz.

5.4.6 Pretprocesorska naredba #include

#include kaže kompilatoru da na njeno mjesto ubaci sadržaj zadane datoteke.

Sintaksa :

```
#include "naziv datoteke"
```

ili

```
#include <naziv datoteke>
```

Primjer :

Sadržaj datoteke "vanjska.cpp" uključujemo u program, čiji izvorni kod se nalazi u datoteci "Ukljuci.cpp" .

Datoteka "Ukljuci.cpp" :

```
#include <iostream.h>
#include "vanjska.cpp"
void main () {
    v_var = 10; // Globalna varijabla
    f_vanjska();
}
```

Datoteka "Vanjska.cpp":

```
int v_var;
void f_vanjska () {
    cout << "Vanjska funkcija" << endl;
}
```

Funkcija f_vanjska poziva se na isti način kao da je uključena u glavnu datoteku.

5.4.7 Standardne biblioteke funkcija

5.4.7.1 iostream.h

Biblioteka *iostream* definira razrede (klase) koje podržavaju rad s ulaznim i izlaznim tokovima.

5.4.7.1.1 Predefinirani objekti tokova C++ :

Ime	Tip razreda	Opis
cin	ulazni tok	Pridružen standardnom ulazu (tipkovnica)
cout	izlazni tok	Pridružen standardnom izlazu (ekran)
cerr	izlazni tok	Pridružen standardnom izlazu u slučaju pogreške (ekran) s izlazom bez korištenja međuspremnika
clog	izlazni tok	Pridružen standardnom izlazu u slučaju pogreške (ekran) s izlazom uz korištenje međuspremnika

5.4.7.2 string.h

5.4.7.2.1 Funkcije za kopiranje

void *memcpy(void *s1, const void *s2, size_t n);

Kopira n znakova s memorijskog područja $s2$ u $s1$. Radi kao strncpy ali ne stane kod null znakova. Ne radi ako se memorijska područja preklapaju.

void *memmove(void *s1, const void *s2, size_t n);

Slično memcpy() ali radi čak i ako se memorijska područja preklapaju.

char *strcpy(char *s1, const char *s2);

Kopira $s1$ u $s2$ tako dugo dok se ne iskopira null znak.

char *strncpy(char *s1, const char *s2, size_t n);

Kao strcpy(), ali kopira najviše n znakova.

5.4.7.2.2 Funkcije spajanja polja znakova

char *strcat(char *s1, const char *s2);

Dodaje kopiju $s2$ na kraj polja $s1$.

char *strncat(char *s1, const char *s2, size_t n);

Kao strcat(), ali dodaje najviše n znakova.

5.4.7.2.3 Funkcije uspoređivanja

int memcmp(const void *s1, const void *s2, size_t n);

Kao strcmp() ali ne stane kod null znakova.

int strcmp(const char *s1, const char *s2);

Uspoređuje argumente i vraća cijeli broj kao rezultat (0 ako je $s1$ jednak $s2$).

int strcoll(const char *s1, const char *s2);

Slično strcmp() ali koristi lokalno definiran poredak znakova umjesto originalnog.

int strncmp(const char *s1, const char *s2, size_t n);

Kao strcmp(), ali uspoređuje najviše n znakova.

size_t strxfrm(char *s1, const char *s2, size_t n);

Transformacija polja znakova. Transformira *s2* u prostor jednak onom kojeg zauzima *s1* tako da se rezultat može uspoređivati s *strcmp()*.

5.4.7.2.4 Funkcije za pretraživanje

void *memchr(const void *s, int c, size_t n);

Kao *strchr()*, ali stane nakon prvih *n* znakova umjesto nakon null znaka.

char *strchr(const char *s, int c);

Vraća pokazivač na prvo pojavljivanje znaka *c* u polju znakova *s*, ili NULL pokazivač ako se *c* ne pojavljuje u polju znakova.

size_t strcspn(const char *s1, const char *s2);

Vraća dužinu početnog segmenta polja znakova *s1* koje se u potpunosti sastoji od znakova koji nisu iz *s2*.

char *strpbrk(const char *s1, const char *s2);

Vraća pokazivač na prvo pojavljivanje polja znakova *s1*, na bilo koji znak iz polja znakova *s2*, ili na NULL pokazivač ako nema znakova iz *s2* koji postoje u *s1*.

char *strrchr(const char *s, int c);

Vraća pokazivač na zadnje pojavljivanje znaka *c* u polju znakova *s*, ili NULL pokazivač ako *c* ne postoji u polju znakova.

size_t strspn(const char *s1, const char *s2);

Vraća dužinu početnog segmenta polja znakova *s1* koja se u potpunosti sastoji od znakova iz polja znakova *s2*.

char *strstr(const char *s, const char *pat);

Vraća pokazivač na prvo pojavljivanje uzorka polja znakova *pat* u *s*.

5.4.7.2.5 Razne funkcije

void *memset(void *s, int c, size_t n);

Postavlja prvih *n* znakova u memorijskom području *s* na vrijednost znaka *c*.

char *strerror(int errnum);

Vraća pokazivač na tekst poruke za zadani broj pogreške.

size_t strlen(const char *s);

Vraća broj znakova u *s*, ne računajući null znak za prekidanje polja znakova.

5.4.7.3 math.h

5.4.7.3.1 Trigonometrijske funkcije

double acos(double x);

Daje arkus arc cos od *x*. Domena: -1 do +1. Raspon: 0 do pi.

`double asin(double x);`

Daje arc sin od x. Domena: -1 do +1. Raspon: $-\pi/2$ do $\pi/2$.

`double atan(double x);`

Daje arc tan od x. Domena: nedefinirana. Raspon: $-\pi/2$ do $\pi/2$.

`double atan2(double x, double y);`

Vrijednost arc tan od y/x , koristi predznak oba argumenta za određivanje kvadranta povratne vrijednosti.

Bilo x ili y mogu biti nula, ali ne oba.

`double cos(double x);`

Kosinus od x, gdje je x u radijanima.

`double sin(double x);`

Sinus x, gdje je x u radijanima.

`double tan(double x);`

Tangens od x, gdje je x u radijanima.

5.4.7.3.2 Eksponecijalne i logaritamske funkcije

`double exp(double x);`

Eksponecijalna funkcija od x.

`double frexp(double x, int *ekspONENT);`

Rastavlja broj x na mantisu i eksponent na bazu 2, tako da je rezultat funkcije mantisa, a eksponent se pohranjuje na mjesto drugog argumenta funkcije.

`double ldexp(double x, int eksponent);`

Računa broj na temelju zadane mantise i eksponenta; inverzna funkcija je frexp.

`double log(double);`

Prirodni logaritam.

`double log10(double);`

Logaritam po bazi 10.

`double modf(double x, double *cijeliDio);`

Rastavlja x na njegov cijeli i decimalni dio.

5.4.7.3.3 Funkcije za potenciranje

`double pow(double baza, double eksponent);`

Potenciranje; argumenti su baza i eksponent.

`double sqrt(double x);`

Kvadratni korijen od x.

5.4.7.3.4 Najbliži cijeli broj, apsolutna vrijednost, i funkcije ostatka dijeljenja

`double ceil(double x);`

Najbliži veći cijeli broj od x.

double fabs(double x);
Apsolutna vrijednost realnog broja x.

double floor(double x);
Najbliži manji cijeli broj od x.

double fmod(double x, double y);
Ostatak dijeljenja realnih brojeva x i y.

5.4.7.4 ctype.h

5.4.7.4.1 Funkcije testiranja znakova

int isalnum(int ch);
ch je alfanumerički znak.

int isalpha(int ch);
ch je slovo.

int iscntrl(int ch);
Provjerava je li znak neki kontrolni znak (ASCII kodovi 0..31 i 127).

int isdigit(int ch);
Provjerava je li ch znamenka (0..9).

int isgraph(int ch);
ch je znak za ispis; kao isprint osim što vraća false za razmak.

int islower(int ch);
ch je malo slovo.

int isprint(int ch);
Provjerava je li ch znak za ispis (ASCII kod između 32 i 126).

int isspace(int ch);
Je li znak praznina (razmak, tab, znak CR, znak za novi red, ili prijelaz na novu stranicu).

int isupper(int ch);
Provjerava da li je ch veliko slovo.

int isxdigit(int ch);
Provjerava da li je ch heksadecimalni broj, 0 - F.

5.4.7.4.2 Funkcije za promjenu malih/velikih slova

int tolower(int ch);
Ako ch predstavlja veliko slovo, rezultat je odgovarajuće malo slovo.

int toupper(int ch);
Ako ch predstavlja malo slovo, rezultat je odgovarajuće veliko slovo.

